Received: Added at production | Revised: Added at production | Accepted: Added at production

DOI: xxx/xxxx

RESEARCH ARTICLE

Declarative Time-Based Animation using Domain Specific Language

Dr Moratanch N[1] | Srikaanth R[2] | Srinivasan S[2] | Priyadharsan K[2]

[1]Assistant Professor, Department of Computer Science and Engineering, SRC, SASTRA Deemed University, Kumbakonam, India

[2]Undergraduate Student, Department of Computer Science and Engineering, SRC, SASTRA Deemed University, Kumbakonam, India

Correspondence
Corresponding to: Dr Moratanch N, Assistant Professor, Department of Computer Science and Engineering, SRC, SASTRA Deemed University, Kumbakonam,India
Email: authorone@gmail.com

Abstract

Animations are an essential element of the current interactive systems, where the visual behavior is supposed to change with time in a known and predictable fashion. Even with the ubiquitous presence of animation tools, time as a representable and analyzable entity is not easily captured especially where the animation behavior needs to be fine-grained. Most of the currently used methods of animation implicitly represent time by discrete keyframes or using percentages, thereby restricting the ability to infer time. To overcome this drawback, it has been inspired by the GameScript, a domain-specific language that minimizes complexity with minimum programming in the creation of 2D games. GameScript represents every production-level object as a self-reliant entity, known as an Agent, and uses a Per- Agent System to control behavior through time. Based on this idea,the given approach proposes an approach, in which time is a first-class function, and time can be evaluated, serialized, and reasoned about in a systematic manner. Within this context, time is evaluated as an unending operation and systematically translated into values to be implemented. At the time analysis step the expressions written in the DSL are interpreted and converted into representations that can be executed by general animation executor. Consequently, the definition of animation is divided into the two issues: temporal and visual realization. The result of the synthesizing is eventually translated into Cascading Style Sheets (CSS) wherein keyframes and animation properties are automatically created. The system does not require any manual working with keyframes and percentage values by using a compiler-like workflow comprising of lexical, syntactic, and semantic analysis. The abstraction enables the expression of complex, time-constrained animations at a higher level, resulting in increased clarity, accuracy and maintainability and compatibility with existing web rendering pipelines.

KEYWORDS

Animation - Domain Specific Language (DSL); Animation Behaviour; Temporal Sampling; Keyframe Generation; Time-Aware Compilation; Declarative Animation Modeling

1 | INTRODUCTION

Animations are ubiquitous in contemporary digital media, appearing in applications ranging from video games to everyday visual displays such as electronic billboards. As technological advancements have evolved, the process of scripting animations has become increasingly streamlined; however, challenges remain, particularly in achieving precise timing control and fine-tuning animation parameters. Current tools, including CSS animations and AI-driven solutions, often struggle with complex timing adjustments or granular control, especially when users require detailed customization. To address these challenges, we propose a simplified approach to creating CSS animations by integrating GameScript domain-specific language (DSL)into the development workflow. This integration facilitates the automatic generation of key CSS components, including @keyframes rules and related properties, thereby enhancing efficiency and accuracy in animation development. Originally, GameScript was designed for creating interactive and visual content such as classic arcade-style games, including Asteroid Destroyer and Tetris. Its application to the web domain leverages its structured syntax, repeatability, and logical clarity. Importantly, we do not suggest that GameScript or our enhancements are intended

to replace traditional CSS coding; rather, they serve as tools that generate CSS code efficiently. Our focus is on enabling smoother curve transitions, hover effects, focus states, active states, group interactions, property transformations, and peer-related logic.While our approach offers significant advantages, it is important to acknowledge certain limitations inherent to the current architecture of GameScript, such as the implementation of grid-based functionalities. We propose a methodology that converts a DSL built by us, characterized by a user-friendly, IF-ELSE structured syntax similar to natural language, into CSS animations through a series of processing stages. These include parsing, sampling, semantic interpretation, and a critical timing layer component. The timing layer plays a pivotal role in controlling the precise output of animations, enabling users to achieve desired effects with improved accuracy and ease.

2 | RELATED WORKS

This section looks at recent research done in game scripting, domain-specific languages, narrative control, machine assistance, and ease-of-use game development methodologies. Rather than separate or individual literature reviews, the discussion follows the continuity of major concepts in the literature as progression is made with respect to works that account for the weaknesses of earlier approaches and how these extensions contribute to the design of the proposed GameScript (GS).

2.1 | Declarative Game Scripting

The early research in the domain of game scripting focused on reducing procedural complexity by representing game logics with rules and declarative constructs. Initial studies showed that from structured rule sets, game mechanics and levels could be generated automatically and that expressive gameplay does not necessarily require detailed procedural coding (1). This work established that rule-based representations would be feasible as a foundation for game logics. As rule-based systems attracted attention, researchers started to investigate their expressive power in practical settings. Investigations on declarative rule-based languages presented that compact formulations of rules can qualitatively record emergent behaviors and complex interactions within games (16). However, as these systems increase in size, concerns move to script organization, efficiency, and maintainability. Further investigation emphasized poorly designed or architected scripting languages that can hamper both development speed and runtime performance (5). Later approaches, to gain clarity and, particularly, long-term maintainability, introduced structured declarative models that explicitly separate game state from behavioral rules (18). Although this separation benefited readability and organization, most of the existing solutions tended to miss a unified and lightweight scripting approach that properly balances simplicity against expressive control. These observations lay the basis for GS, which uses a compact, rule-based scripting approach that is intended to remain readable even as game logic becomes more complex.

2.2 | DSL for Game Development

Based on the shortcomings of general-purpose scripting, there were efforts directed at domain-specific languages(DSLs) as a solution to increase the level of abstraction in game development. Some of the initial attempts in the field of DSLs were the incorporation of game-specific abstractions into host languages, allowing the description of behavior on multiple abstraction levels, yet using the existing execution environment (10). This was followed by further research work in which extensible game description languages were developed to make changes in game rules and behavior programmable through these languages without requiring changes in game engines (13). Although this made game development more flexible, this area of research again emphasized the need for well-defined domain concepts to prevent over-engineering. With further advances in DSL research, more emphasis was given to serious games like educational and games for educational institutions. DSL for modeling adventure and educational games successfully applied domain-specific concepts to define gameplay (18). Further evolution to the language-driven development demonstrated that the definition of game logic before the engine construction allows achieving better modularity and reusability of the code (15). However, most DSLs still have steep learning curves or rigid syntax. GS addresses this gap by offering a minimal, rule-oriented DSL that maintains abstraction advantages and stays comprehensible and easy to modify.

2.3 | Engine Architecture

Game scripting development also incorporates narrative-oriented and interactive story solutions. Structured game scripts have been found early on in research to facilitate branching scenarios and conversations, making it feasible for any individual to develop interactive storylines without

necessarily being programmatically competent (4). These hypotheses were later verified by game implementations where the scripts handled conversations, puzzle-solving, and storylines in adventure-type games (7). With the rising interest in Narrative Automation, researchers did look into AI solutions. Some recent work used Large Language Models to automatically create an interactive story sequence (21). But this work posed its own problems regarding predictability and execution. Some related work examined properties in game dialogues (22), while others explored ways to model behavioral data to handle non-linear narratives (25). Although these approaches make progress for narrative flexibility, they frequently do so at the expense of transparency and simplicity. By contrast, GS focuses on explicit, rule-based scripting of narratives, giving designers more control over narrative logic without resorting to complex AI-intensive systems.

2.4 | Story Scripting Solution

Game scripting development also incorporates narrative-oriented and interactive story solutions. Structured game scripts have been found early on in research to facilitate branching scenarios and conversations, making it feasible for any individual to develop interactive storylines without necessarily being programmatically competent (4). These hypotheses were later verified by game implementations where the scripts handled conversations, puzzle-solving, and storylines in adventure-type games (7). With the rising interest in Narrative Automation, researchers did look into AI solutions. Some recent work used Large Language Models to automatically create an interactive story sequence (21). But this work posed its own problems regarding predictability and execution. Some related work examined properties in game dialogues (22), while others explored ways to model behavioral data to handle non-linear narratives (25). Although these approaches make progress for narrative flexibility, they frequently do so at the expense of transparency and simplicity. By contrast, GS focuses on explicit, rule-based scripting of narratives, giving designers more control over narrative logic without resorting to complex AI-intensive systems.

2.5 | Scripting for Education

The requirement for simple scripting tools is particularly important in education-based as well as gamification-based systems. The initial research proved that knowledge modeling through scripting can be useful in structuring gamebased learning activities in education-based games (3). Simplified scripting engines with a reduced instruction set were later developed to facilitate scripting activities even for beginners, allowing students to conduct scripting activities in games without having in-depth knowledge in programming (6). On top of this, the development of authoring systems followed to enable the creation of gamified learning activities by high-level scripting (8). Research continued with collaborative learning, team evaluation, and games for inquiry-based learning (24, 26, 27, 30). Although the emphasis of these studies now lies on pedagogical outcome rather than technical design, the significance of understandable representations of game logic remains, thus upholding the importance of GS.

2.6 | Visual Design Methods

Alongside script-based solutions, model-based and graphical abstractions have been investigated to further minimize development time. Domain-specific modeling languages and model-driven development allowed showing the capability to transform high-level models directly into game executables with less reliance on programming (17, 19). Graph DSLs furthered this concept to enable designers to represent game concepts in graphical forms (20). Even as they provide strong abstraction over low-level details, they can be rather restrictive in terms of expressivity. Such a trade-off between abstraction and expressibility serves to emphasize GS as a middle path that combines scripting expressibility with simplicity.

2.7 | Game Design Planning

Some research concentrates on supporting conceptual and nascent designs as opposed to executable logic. Game sketching assists in quickly evaluating designs (23), and game design documents aid in maintaining consistency and theme integrity (29). It can be seen that these contributions are useful for the purpose of planning, but they do not cover the specification and implementation of game behavior. Lastly, research on the use of blockchain and game theory for federated learning incentivization mechanisms (Paper 12) can be considered irrelevant to this project since it neither relates to game scripting nor to methodologies on developing games.

2.8 | Comparision

No currently existing model is similar to our system because what we have implemented is a combination of a set of features that can be observed in multiple models, and each of them independently achieves animations by different means. As far as we understand, none of the existing systems is able to combine these features in a single compile-time domain-specific language (DSL) specifically built to create Tailwind-compatible CSS animations.

Key Points

- Previous work involved rule-based and declarative scripting to alleviate procedural complexity while preserving expressive game behavior.
- Domain-specific languages enhanced abstraction in game development but tended to impose inflexible syntax or learning overhead.
- Game scripting engines concentrated on efficiency and modularity, sometimes adding architectural complexity.
- Narrative and educational scripting paradigms prioritized usability but compromised transparency and predictability.
- Model-driven and visual approaches made design simpler but restrictive in expression, thus underlining the need for a lightweight scripting solution.

3 | PROPOSED METHODOLOGY

We introduce a conceptual framework that will make the creation of animations easier and allow performing more granular control over animation parameters. The methodology entails the consumption of the temporal information as one of the essential input parameters in a domain-specific language (DSL) syntax that comprises the full descriptive semantics of animation sequence desired. This DSL input is then manipulated and converted through a methodical conversion process to machine readable and understandable numerical values in accordance with CSS property values. The transformed values are then systematically injected into the CSS rendering pipeline, thus, producing the animation with built-in time control, which in turn is able to provide the ability to time, sequence and synchronise control mechanisms that are part of the animation cycle.

3.1 | Temporal Analysis

The Domain-Specific Language (DSL) requires the end-user to define a temporal parameter that establishes the specific timelines in which the action should be executed, and the specific set of operations that should be instantiated at such timelines. It shares the underlying computational logic of a conditional (if-then) construct whereby the active modulation of the animation parameters is the predicated parameter, that is, the parameter of opacity, based on continuous and periodic functions represented as trigonometric sine and cosine functions. The choice of these functions is based on their intrinsic simplicity, periodicity and determinism that, together, allow the production of repeatable and predictable visual effects needed to produce consistent animation effects in graphical user interfaces. Opacity, represented as a time-varying constant, O(t) is represented as a continuous function of time t, and is gradually increasing as the animation advances through the animation phase. All these functions make transitions between visual levels of transparency smooth, and they are the workings of developing the animated element within the temporal domain. Other functions like tangent are not included on purpose because of their predisposition to asymptotic discontinuities and unstable behavior that cannot be trusted to maintain animation fidelity. Even though linear interpolation models may be used, it is possible that the interpolation may create sharp or abrupt transitions hence compromising visual smoothness. Although the sine and cosine functions form the main modeling technique, more complicated parametric curves, including Bezier splines, can be combined in the case they are needed to provide more detailed or stylistically rich animation effects. The temporal profile T (t) obtained through a sine-based calculation provides a parameterizable temporal profile that can be directly inserted into Cascading Style Sheets (CSS) animation keyframes and transition definitions, compatible with web-based rendering engines and making it easy to control the transitions between animated changes in opacity.

3.2 | System Computational

The Domain-Specific Language (DSL) requires the end-user to define a temporal parameter that establishes the specific timelines in which the action should be executed, and the specific set of operations that should be instantiated at such timelines. It shares the underlying computational logic of a conditional (if-then) construct whereby the active modulation of the animation parameters is the predicated parameter, that is, the

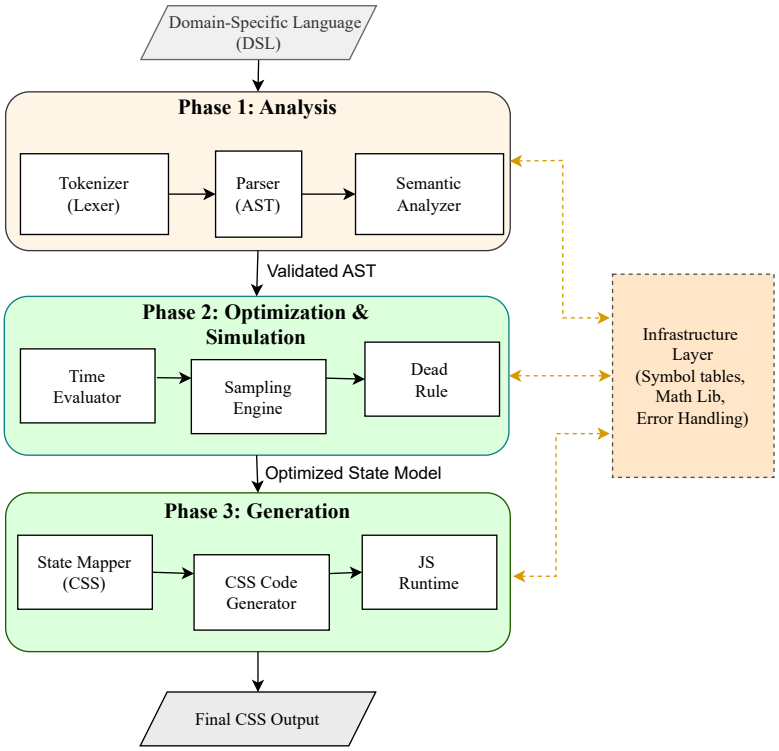

FIGURE 1 Architecture diagram of the GameScript compilation pipeline

parameter of opacity, based on continuous and periodic functions represented as trigonometric sine and cosine functions. The choice of these functions is based on their intrinsic simplicity, periodicity and determinism that, together, allow the production of repeatable and predictable visual effects needed to produce consistent animation effects in graphical user interfaces. Opacity, represented as a time-varying constant, O(t) is represented as a continuous function of time t, and is gradually increasing as the animation advances through the animation phase. All these functions make transitions between visual levels of transparency smooth, and they are the workings of developing the animated element within the temporal domain. Other functions like tangent are not included on purpose because of their predisposition to asymptotic discontinuities and unstable behavior that cannot be trusted to maintain animation fidelity. Even though linear interpolation models may be used, it is possible that the interpolation may create sharp or abrupt transitions hence compromising visual smoothness. Although the sine and cosine functions form the main modeling technique, more complicated parametric curves, including Bezier splines, can be combined in the case they are needed to provide more detailed or stylistically rich animation effects. The temporal profile T (t) obtained through a sine-based calculation provides a parameterizable temporal profile that can be directly inserted into Cascading Style Sheets (CSS) animation keyframes and transition definitions, compatible with web-based rendering engines and making it easy to control the transitions between animated changes in opacity.

4 | PROCESSING PIPELINE

The architecture of the proposed system is described in this section. It is designed in a classical compiler pipeline framework, except that it is made to accept time-based animation specifications in the form of a domain-specific language. The system takes a DSL input and analyzes it at the front-end stage of analysis, optimizes it with time-awareness, and produces animation code. The architecture is divided into three key steps, namely, the front-end processing, time evaluation optimization, and code generation in the back-end. The stages are tasked with the responsibility of gradually converting the input specification into a form that can be executed.

4.1 | Domain-Specific Language

A domain-specific language is used as the input of the proposed system. The DSL is intended to describe animation behavior through time in a explicit manner, which is hard to describe using the standard constructs of CSS animation. The language permits the user to define time intervals, define state transitions in between time intervals, and states the connections among various animation elements, e.g. sequential or parallel execution. The DSL is declarative and thus the system can reason about animation behavior without having to make reference to the details at the implementation level.

4.2 | DSL Program Analysis Pipeline

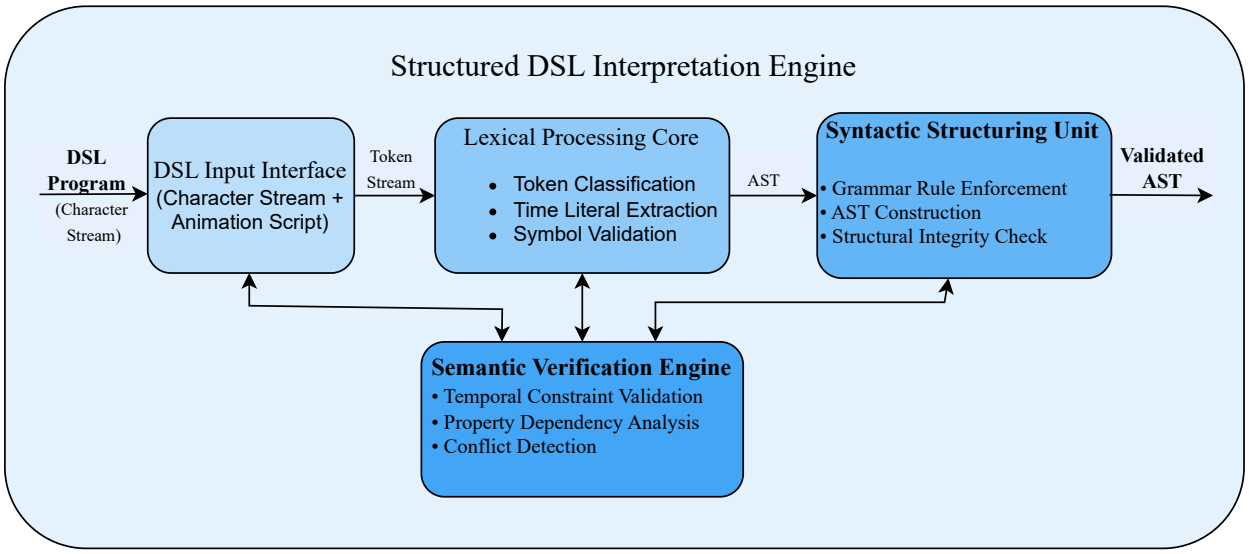

FIGURE 2 Architecture diagram of Structured DSL Interpretation Engine

The input DSL program is processed through a structured validation mechanism that ensures both syntactic correctness and temporal consistency. As illustrated in Figure 2, the process begins with lexical analysis, where the DSL input is interpreted as a stream of characters and transformed into a sequence of tokens, including time literals, identifiers, keywords, and operators. This stage enforces lexical compliance and detects invalid symbols. The syntactic analysis stage constructs an Abstract Syntax Tree (AST) that captures the hierarchical structure of animation specifications while validating grammar rules and detecting improper nesting or missing constructs. Subsequently, semantic validation verifies logical consistency across time intervals, animation properties, and state dependencies. Conflicting transformations over overlapping time ranges are identified and annotated. The result of this phase is a validated and temporally consistent intermediate representation that forms the foundation for deterministic scheduling in subsequent stages.

4.3 | Temporal Modeling And Optimization

Following semantic validation, the system advances to deterministic temporal scheduling and state consolidation, where temporal specifications are interpreted and reconciled into a unified execution model. As depicted in Figure 3, the time evaluator normalizes and scales declared time intervals, constructing a coherent global timeline across all animation components. Continuous temporal functions are discretized by the sampling engine, which estimates animation states at fixed intervals to generate executable state frames suitable for CSS keyframes. Concurrent transformations affecting identical properties are examined for temporal overlap, and conflicts are resolved using predefined precedence rules to ensure that only one valid transformation governs any given time instant. Redundant or unreachable transitions are pruned through rule consolidation mechanisms. This phase preserves DSL semantics while enforcing deterministic behavior and ensuring temporal coherence across concurrent animation specifications.

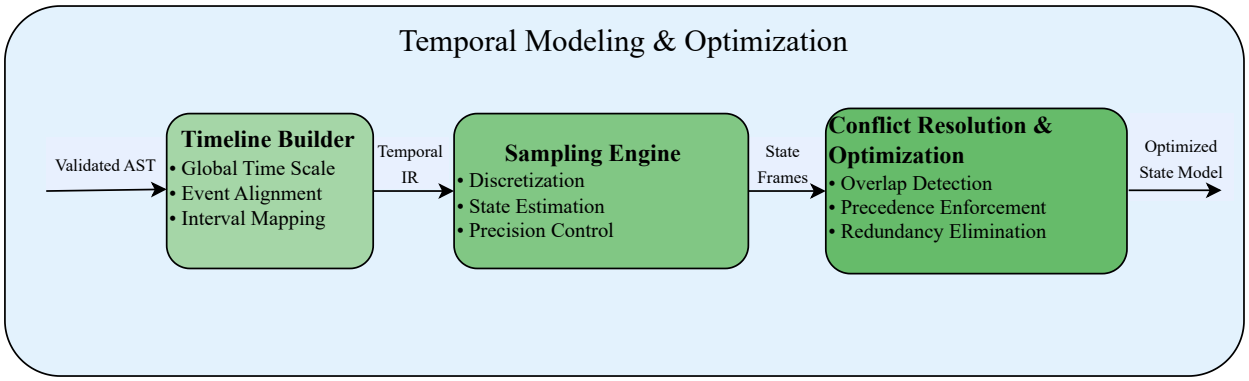

FIGURE 3 Architecture diagram of Temporal Modeling and Optimization

4.4 | CSS Synthesis and Runtime Integration

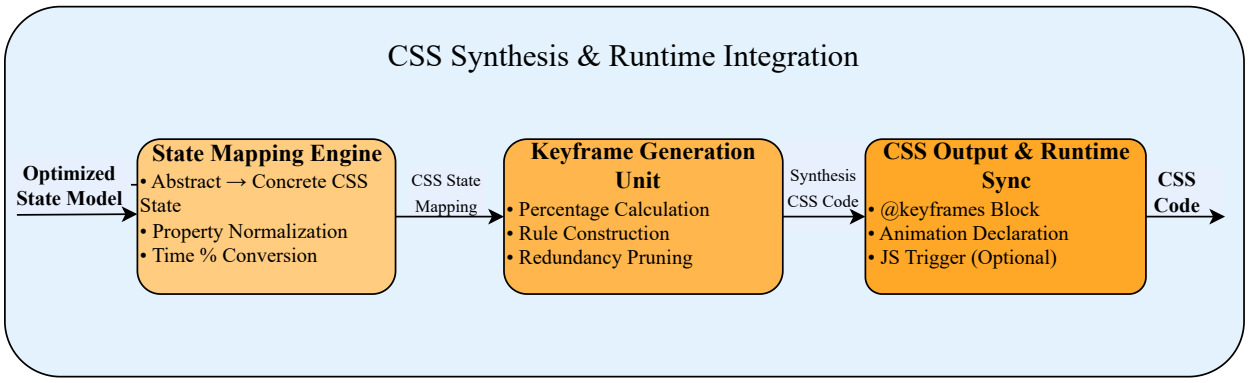

FIGURE 4 Architecture diagram of CSS Synthesis and Runtime Integration

In the final stage, the optimized temporal model is translated into executable animation definitions compatible with web rendering environments. As shown in Figure 4, abstract state representations are mapped to concrete CSS properties through a structured state-mapping mechanism that preserves the temporal relationships established during scheduling. The system generates synthetic CSS keyframes and animation declarations that accurately reflect the discretized timeline. Redundant keyframes are eliminated to ensure compactness while maintaining visual smoothness and temporal fidelity. Where dynamic triggering or synchronization is required, lightweight runtime coordination integrates the generated CSS output without duplicating its functionality. This phase ensures that declarative temporal specifications are faithfully materialized into efficient, standards-compliant animation artifacts.

Key Points

- The system follows a compiler-inspired pipeline that processes the DSL input through lexical, syntactic, and semantic analysis, producing an annotated representation of animations.

- Time evaluation and sampling scale and discretize animation timelines, translating continuous behavior into discrete states suitable for CSS keyframes while maintaining temporal accuracy.
- Optimization rules reduce redundancy, resolve overlaps, and combine compatible animation segments, enhancing performance while preserving DSL semantics.
- The code generation and runtime support convert the optimized representation into CSS-compatible states, keyframes, and dynamic animation management, ensuring efficient, accurate, and maintainable execution.

5 | IMPLEMENTATION

The implementation of the proposed system is centered around a set of core algorithms that operate on the validated intermediate representation produced by the DSL analysis pipeline. These algorithms handle temporal ordering, timeline construction with conflict resolution, and CSS keyframe generation.

5.1 | Implemented Algorithms

The following algorithms describe the core implementation stages of the proposed system.

Algorithm 1

Since animation events may be declared in arbitrary order within the DSL, chronological ordering is required to ensure deterministic execution.

Algorithm 1 Temporal Event Ordering

float

Require: Set of animation events $E = \{e_1, e_2, ..., e_n\}$ with associated time values
Ensure: Chronologically ordered event list E'
for all $e_i \in E$ **do**
    Extract timestamp t_i from e_i
end for
Sort E in ascending order of t_i using a stable sorting strategy
$E' \leftarrow E$

Result:
return E'

Deterministic temporal ordering is achieved regardless of the declaration order in the DSL. Events with identical timestamps preserve their relative order, ensuring predictable execution for both sequential and parallel animations.

Algorithm 2

Algorithm 2 Timeline Construction and Conflict Resolution

float

Require: Temporally ordered event list E'
Ensure: Conflict-free property timeline $\{T_p\}$
for all animation properties p **do**
  Initialize an empty timeline T_p
end for
for all *event* $e \in E'$ **do**
  Convert e into a time interval $[t_{start}, t_{end}]$
  Insert the interval into the corresponding T_p
  if overlapping intervals are detected in T_p **then**
    Resolve conflicts using predefined precedence rules
    Merge or trim intervals accordingly
  end if
end for

Result:
return $\{T_p\}$

Overlapping transformations applied to the same property are resolved deterministically, ensuring that a single valid transformation is active at any given time instant and preventing unstable animation states.

Algorithm 3

Continuous temporal semantics must be converted into discrete CSS keyframes; hence normalization and sampling are necessary.

Algorithm 3 CSS Keyframe Generation

float

Require: Optimized timelines $\{T_p\}$ and total animation duration D
Ensure: CSS keyframes and corresponding animation declarations
for all time slice t_i in T_p **do**
  for all state change occurring at time t_i in T_p **do**
    Normalize time: $k \leftarrow (t_i/D) \times 100$
    Generate a keyframe at k% with the corresponding property value
  end for
end for
Eliminate redundant keyframes
Emit CSS keyframes definitions and animation rule

Result:
return Generated CSS code

Compact and valid CSS animations are generated automatically. Redundant keyframes are removed while preserving visual smoothness and accurately reflecting the temporal semantics of the DSL.

6 | DATASETS

As the proposed approach does not rely on an existing deterministic benchmark dataset, evaluation is conducted using a curated set of representative animation tasks. These tasks model common time-based user interface animation behaviors and are used as structured inputs at the specification level for validating the proposed domain-specific language. The dataset is designed to capture fundamental animation patterns involving temporal progression, easing, concurrency, and repetition.

Fade-in over time. The element remains initially invisible and transitions to a fully visible state over a fixed duration. The initial opacity is set to 0 and progressively increases to 1 over a duration of 1 second.

```
over 1s:
  opacity 0 -> 1
```

Horizontal translation with easing. An element moves horizontally along the X-axis using a smooth ease-in-out timing function. The animation starts at position x = 0 and translates to x = 100 pixels over a duration of 2 seconds.

```
over 2s ease-in-out:
  translateX 0 -> 100
```

Scale with opacity. An element simultaneously scales and changes opacity. The scale increases from 0.8 to 1.0 while the opacity transitions from 0 to 1 over a duration of 1.5 seconds.

```
over 1.5s:
  scale 0.8 -> 1
  opacity 0 -> 1
```

Staggered animation. Multiple elements animate sequentially with a fixed temporal offset between their start times. Each element animates over 1 second, with a stagger delay of 0.2 seconds, using vertical translation and opacity effects.

```
stagger 0.2s:
  over 1s:
    opacity 0 -> 1
    translateY 20 -> 0
```

Looping animation with delay. An animation is repeated indefinitely with a fixed delay between successive iterations. Each iteration performs a full rotation from 0 to 360 degrees over 1 second, followed by a delay of 0.5 seconds.

```
loop infinite:
  over 1s:
    rotate 0 -> 360
  delay 0.5s
```

These representative tasks collectively evaluate the expressiveness and correctness of the proposed DSL across single-property animations, multi-property synchronization, easing behavior, staggered execution, and looping constructs. This task-based dataset enables systematic validation of temporal reasoning, conflict resolution, and CSS code generation without requiring external animation benchmarks.

REFERENCES

[1] Khalifa A. Automatic game script generation. Master's thesis. Cairo University, Faculty of Engineering; 2015.

[2] Wu Y, Yao X, He J. An innovation of the game script engine development based on J2ME multimedia mobile device. In: Proceedings of the 2011 International Conference on Computer Science and Service System (CSSS). IEEE; 2011:193–195.

[3] Fatimah DDS. Script knowledge representation in game designing for instructional media. *IOP Conf Ser Mater Sci Eng*. 2018;434(1):012053.

[4] Engström H. 'I have a different kind of brain'—a script-centric approach to interactive narratives in games. *Digital Creativity*. 2019;30(1):1–22.

[5] White W, Koch C, Gehrke J, Demers A. Better scripts, better games: smarter, more powerful scripting languages will improve game performance while making gameplay development more efficient. *Queue*. 2008;6(7):18–25.

[6] Steffen M, Zambreno J. Exposing high school students to concurrent programming principles using video game scripting engines. In: Proceedings of the 2012 ASEE Annual Conference & Exposition; 2012:25–623.

[7] Song W, Huang L, Liu H, Zhao S, Wu L, Shu H. Design and development of 2D picture book style text-based adventure game. In: Proceedings of the 2022 International Conference on Computer Graphics, Image and Virtualization (ICCGIV). IEEE; 2022:63–66.

[8] Feng-Jung L, Chia-Mei L. Design and implementation of a collaborative educational gamification authoring system. *Int J Emerg Technol Learn*. 2021;16(17):277.

[9] Yue X. Development and implementation of Java game engine based on network information technology. *J Phys Conf Ser*. 2021;1992(3):032108.

[10] Calleja A, Pace GJ. A domain-specific embedded language approach for the scripting of game artificial intelligence. University of Malta, Faculty of ICT; 2009.

[11] Öztürk S. Analyzing maintainability factors in open-source game engines: implications for game developers. *Multimed Tools Appl*. 2025;84(35):43929–43957.

[12] Zuo C, Xu P, Song Y, Lu J, Yuan C, Li Y. RAIM: three-stage Stackelberg game for hierarchical federated learning with reputation-aware incentive mechanism. *Sci Rep*. 2025;15(1):34344.

[13] Schaul T. An extensible description language for video games. *IEEE Trans Comput Intell AI Games*. 2014;6(4):325–331.

[14] Zahari AS, Ab Rahim L, Nurhadi NA, Aslam M. A domain-specific modelling language for adventure educational games and flow theory. *Int J Adv Sci Eng Inf Technol*. 2020;10(6).

[15] Moreno-Ger P, Martínez-Ortiz I, Sierra JL, Manjón BF. Language-driven development of videogames: the experience. In: International Conference on Entertainment Computing. Springer; 2006:153–164.

[16] Julia C, van Rozen R. ScriptButler serves an empirical study of PuzzleScript: analyzing the expressive power of a game DSL through source code analysis. In: Proceedings of the 18th International Conference on the Foundations of Digital Games; 2023:1–11.

[17] Tang S, Dennett C. Towards a domain specific modelling language for serious game design; 2008.

[18] White W, Sowell B, Gehrke J, Demers A. Declarative processing for computer games. In: Proceedings of the 2008 ACM SIGGRAPH Symposium on Video Games; 2008:23–30.

[19] Thillainathan N. A model driven development framework for serious games. In: INFORMATIK 2013—Informatik angepasst an Mensch, Organisation und Umwelt; 2013:81–92.

[20] Sanchez P, Domínguez M. Hernández. *Marta y García, Y*. 2008.

[21] Kumaran V, Rowe J, Mott B, Lester J. Scenecraft: automating interactive narrative scene generation in digital games with large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment. 2023;19(1):86–96.

[22] Schlenk M, Efer T, Burghardt M. Treating games as plays? Computational approaches to the detection of scenes in game dialogs. In: CHR; 2024:1128–1138.

[23] Agustin M, Chuang G, Delgado A, Ortega A, Seaver J, Buchanan JW. Game sketching. In: Proceedings of the 2nd International Conference on Digital Interactive Media in Entertainment and Arts; 2007:36–43.

[24] Jing-Li H, Hung-Yang S. "Game playing" and "game making": gamified applications of topical education. In: International Conference on Computers in Education; 2020.

[25] Cianciulli S, Riccardelli D, Vassos S. Coordinating dialogue systems and stories through behavior composition. In: Workshop on Computer Games. Springer; 2014:150–163.

[26] Farah YA, Dorneich MC. Development of a gamified cooperative teamwork environment using an event-based approach. In: Extended Abstracts of the 2022 Annual Symposium on Computer-Human Interaction in Play; 2022:196–202.

[27] Leo DH, Fernández EDV, Pérez JIA, Dimitriadis Y, Lorenzo MLB, Abellán IMJ. The added value of implementing the Planet Game scenario with Collage and Gridcole. 2008.

[28] Ampatzoglou A, Chatzigeorgiou A. Evaluation of object-oriented design patterns in game development. *Inf Softw Technol*. 2007;49(5):445–454.

[29] Rahmadi L, Prambayun A. Design of game design document as an interactive media to introduce the culture of Pagar Alam City. *J Crit Rev*. 2019;6(5):243–251.

[30] Yeoh CP, Li CT, Hou HT. Game-based collaborative scientific inquiry learning using realistic context and inquiry process-based multidimensional scaffolding. *Int J Sci Educ*. 2025;47(8):961–983.

[31] Yar A, Idani A, Ledru Y, Collart-Dutilleul S. Visual animation of B specifications using executable DSLs. In: Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings; 2022:617–626.

[32] Alharbi M, Alshayeb M. Automatic code generation techniques: a systematic literature review. *Autom Softw Eng*. 2026;33(4).

[33] Dewi ENF, Rachman AN, Hartadji RHZ. Implementation of hierarchical finite state machine for controlling 2D character animation in action video game. In: Proceedings of the 2023 Eighth International Conference on Informatics and Computing (ICIC); 2023:1–6.

[34] Lee J, Cho H. Hierarchical finite state machines for real-time game character control. In: IEEE International Conference on Consumer Electronics. IEEE; 2011:325–326.

[35] Harel D. Statecharts: a visual formalism for complex systems. *Sci Comput Program*. 1987;8(3):231–274.

[36] Gregory J. An architecture for data-driven game engines. In: IEEE Game Developers Conference. IEEE; 2009.

[37] Schmidt DC. Model-driven engineering for interactive applications. *Softw Syst Model*. 2006;5(2):131–146.

[38] Colledanchise M, Ogren P. A survey of behavior trees in video games. *IEEE Trans Games*. 2016;8(4):344–357.

[39] Wagner S. Event-driven architecture for game development. *Softw Eng Notes*. 2012;37(4):1–6.

[40] Erdweg S, et al. The state of the art in domain-specific language engineering. *ACM Comput Surv*. 2013;47(3):1–37.